PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ

Arquitetura e Soluções em Cloud

TDE 3 — Vertical Slice + Clean Architecture

Testes unitários de arquitetura com ArchUnitTS nos backends

Projeto: Veterano. — Plataforma de Mentoria entre Veteranos e Calouros (PJBL)

`github.com/ViniMTrevisan/pjbl-mentoria-cloud`

Integrante	Participação
Bento Barp	Grupo do projeto
Guilherme Reis	Grupo do projeto
Guilherme Selenko	Grupo do projeto
Vinicius Trevisan	Grupo do projeto

Curitiba, 26 de outubro de 2026

1. Resumo

Esta entrega aplica dois estilos arquiteturais complementares nos quatro projetos backend do sistema — BFF, mentores-service, sessoes-service e azure-function — e prova a aderência com testes unitários de arquitetura usando o ArchUnitTS, a biblioteca equivalente ao ArchUnit do Java para o ecossistema Node/TypeScript.

A organização anterior era em camadas horizontais: domínio, casos de uso, infraestrutura, HTTP. Depois da entrega, cada capacidade de negócio é uma pasta autocontida (vertical slice) e, dentro dela, as camadas da clean architecture aparecem na ordem domain → application → presentation/infrastructure. O resultado prático é que uma mudança em "avaliação de mentor" ou "outbox de eventos" acontece em uma pasta, com fronteira verificada por teste automatizado.

Projeto backend	Vertical slices	Testes de arquitetura	Total de testes
`bff`	agregacao, mentores, sessoes, eventos, saude	26	35
`mentores-service`	mentores, avaliacoes	8	35
`sessoes-service`	sessoes, outbox	8	35
`azure-function`	reputacao, eventos, saude	12	21
`Total`	12 slices	54	126

Todos os testes e builds (tsc) executam sem falha, conforme os prints da seção 6.

2. Estrutura aplicada

A regra usada em todos os projetos é sempre a mesma. O aplicativo é dividido em fatias verticais por capacidade de negócio; cada fatia repete as camadas por dentro, e só aponta "para dentro" (em direção ao domínio). O que é transversal — erro base, configuração, contratos e helpers HTTP — fica em src/shared, que não pode conhecer nenhuma fatia.

Camada	Responsabilidade	Pode depender de
`domain`	Regras de negócio puras: entidades, validações, máquina de estados, erros.	Nada além do próprio domínio e do kernel compartilhado.
`application`	Casos de uso e portas (interfaces de saída). Orquestra o domínio.	domain, outras portas e shared/application.
`infrastructure`	Implementa as portas: repositórios Mongo/SQL/memória, clients HTTP.	application, domain e shared.
`presentation`	Traduz HTTP em chamadas de caso de uso. Nenhuma regra de negócio.	application, domain e shared/http.

```
projeto/
  src/
    features/              # vertical slices (1 pasta por capacidade)
      <slice-a>/
        domain/            # entidades, erros, regras puras
        application/       # casos de uso + portas (interfaces)
        infrastructure/    # adaptadores: Mongo, SQL, HTTP
        presentation/      # rotas Express / handlers HTTP
      <slice-b>/           # mesma estrutura, independente
    shared/                # kernel: DomainError, config, contratos
    app.ts                 # composition root (injeta adaptadores)
    server.ts              # bootstrap do processo
  test/
    architecture.test.ts   # regras ArchUnitTS
    *.test.ts              # dominio e casos de uso
  vitest.config.ts         # globals: true (exigido pelo ArchUnitTS)
```

3. Aplicação em cada projeto

3.1 mentores-service

Dois slices. O slice mentores concentra o agregado (CRUD, busca com filtros, perfil público) e o slice avaliacoes cuida da nota do mentor, que é uma capacidade separada e tem sua própria regra de duplicidade.

Slice	Camadas	Conteúdo principal	Porta / adaptadores
`mentores`	todas as 4	Mentor, validação, factory, filtros, rotas /mentores	MentorRepository → MongoMentorRepository, InMemoryMentorRepository
`avaliacoes`	sem infra	adicionarAvaliacao, rotas POST /mentores/:id/avaliacoes	usa MentorRepository do slice mentores

Legenda: "todas as 4" = domain, application, infrastructure e presentation; "sem infra" = domain, application e presentation; "só aplicação/apresentação" = apenas essa camada.

```
mentores-service/src/features/mentores/
  domain/          mentor.ts, validation.ts,
                   mentorFactory.ts, errors.ts
  application/     MentorRepository.ts, createMentor.ts,
                   listMentores.ts, getMentor.ts,
                   updateMentor.ts, deleteMentor.ts
  infrastructure/  MongoMentorRepository.ts,
                   InMemoryMentorRepository.ts, mentoresSeed.ts
  presentation/    mentoresRoutes.ts
```

3.2 sessoes-service

O slice sessoes dono do agregado Sessao e da máquina de estados; o slice outbox é a capacidade de publicar eventos de domínio. O mesmo adaptador de banco implementa as duas portas (SessaoRepository e OutboxRepository), porque a gravação do evento é transacional com a mudança de status.

Slice	Camadas	Conteúdo principal	Porta / adaptadores
`sessoes`	todas as 4	Sessao, stateMachine (aceitar/recusar/concluir/cancelar), validação, rotas /sessoes	SessaoRepository → SqlSessaoRepository (Azure SQL) e InMemorySessaoRepository
`outbox`	só aplicação	OutboxRepository, relayOutbox (at-least-once, tolerante a falha do sink)	OutboxRepository → mesma implementação SQL/memória

```
sessoes-service/src/features/sessoes/
  domain/          sessao.ts, stateMachine.ts, events.ts,
                   validation.ts, errors.ts, texto.ts
  application/     SessaoRepository.ts, createSessao.ts,
                   listSessoes.ts, getSessao.ts, changeStatus.ts
  infrastructure/  SqlSessaoRepository.ts,
                   InMemorySessaoRepository.ts, schema.ts
  presentation/    sessoesRoutes.ts
```

3.3 bff

O BFF tem um slice de negócio de verdade (agregacao, com domínio, portas e clients HTTP) e fatias de borda por recurso exposto ao front-end. As fatias de proxy não conhecem adaptador concreto: dependem da porta ServicoRemoto, definida em shared/application, e o ServicoProxy é injetado no composition root.

Slice	Camadas	Conteúdo principal	Porta / adaptadores
`agregacao`	todas as 4	getAggregatedData com degradação parcial, timeout por dependência e limite de concorrência	MentoresPort, SessoesPort, ReputacaoPort → clients HTTP (fetch)
`mentores`	só apresentação	proxy do contrato de mentores	ServicoRemoto

Slice	Camadas	Conteúdo principal	Porta / adaptadores
`sessoes`	só apresentação	proxy do contrato de sessões	ServicoRemoto
`eventos`	só apresentação	proxy do sink de eventos da Function	ServicoRemoto
`saude`	só apresentação	/health agregando as três dependências	ServicoRemoto

3.4 azure-function

A Function é pequena e não tem camada de infraestrutura própria (o store de eventos é em memória, na aplicação). Ainda assim foi dividida em fatias, com o handler na borda e o domínio no centro, para manter a mesma regra de dependência dos demais backends.

Slice	Camadas	Conteúdo principal	Observação
`reputacao`	sem infra	calcularReputacao, média, selo, distribuição, handler POST /api/reputacao	domínio sem imports
`eventos`	sem infra	eventStore em memória com idempotência, handler GET/POST /api/eventos	usa o tipo Reputacao do slice reputacao
`saude`	só apresentação	GET /api/health	—

4. Evidências de código-fonte (GitHub)

Todo o código está no repositório público do grupo. O trabalho foi entregue na branch tde3-vertical-slice-archunit, a partir da main, e está aberto como pull request.

Item	Link
Pull request da entrega	https://github.com/ViniMTrevisan/pjbl-mentoria-cloud/pull/1
Estrutura do bff	https://github.com/ViniMTrevisan/pjbl-mentoria-cloud/tree/tde3-vertical-slice-archunit/bff/src/features
Estrutura do mentores-service	https://github.com/ViniMTrevisan/pjbl-mentoria-cloud/tree/tde3-vertical-slice-archunit/mentores-service/src/features
Estrutura do sessoes-service	https://github.com/ViniMTrevisan/pjbl-mentoria-cloud/tree/tde3-vertical-slice-archunit/sessoes-service/src/features
Estrutura da azure-function	https://github.com/ViniMTrevisan/pjbl-mentoria-cloud/tree/tde3-vertical-slice-archunit/azure-function/src/features
Teste de arquitetura do bff	https://github.com/ViniMTrevisan/pjbl-mentoria-cloud/blob/tde3-vertical-slice-archunit/bff/test/architecture.test.ts
Teste de arquitetura do mentores-service	https://github.com/ViniMTrevisan/pjbl-mentoria-cloud/blob/tde3-vertical-slice-archunit/mentores-service/test/architecture.test.ts
Teste de arquitetura do sessoes-service	https://github.com/ViniMTrevisan/pjbl-mentoria-cloud/blob/tde3-vertical-slice-archunit/sessoes-service/test/architecture.test.ts
Teste de arquitetura da azure-function	https://github.com/ViniMTrevisan/pjbl-mentoria-cloud/blob/tde3-vertical-slice-archunit/azure-function/test/architecture.test.ts

Exemplo de código-fonte do composition root do mentores-service, onde o adaptador concreto é escolhido e injetado nas rotas — nenhuma camada interna conhece Mongo ou Express além dele:

```
// mentores-service/src/app.ts (trecho)
app.use(saudeRoutes("mentores", "banco", () => repo.verificarSaude()));
app.use("/mentores", mentoresRoutes(repo));
app.use("/mentores", avaliacoesRoutes(repo));
```

E o mesmo padrão no BFF, com as portas da agregação satisfeitas por clients HTTP:

```
// bff/src/app.ts (trecho)
const mentores: MentoresPort = new MentoresHttpClient(httpMentores);
const sessoes: SessoesPort = new SessoesHttpClient(httpSessoes);
const reputacao: ReputacaoPort = new ReputacaoHttpClient(httpFunction);
return {
  agregacao: { mentores, sessoes, reputacao },
  proxyMentores: new ServicoProxy(httpMentores, "mentores"),
};
```

5. Testes unitários de arquitetura (ArchUnitTS)

O ArchUnitTS é o equivalente do ArchUnit para TypeScript. A ferramenta analisa o grafo de imports a partir do tsconfig e valida regras declarativas. Nesta entrega foi usada a versão 2.5.4 (pacote archunit), integrada ao Vitest já existente em cada projeto.

Item	Configuração
Pacote	archunit 2.5.4 (devDependency)
Runner	Vitest 2.x com globals: true em vitest.config.ts (exigido pelo ArchUnitTS)
Arquivo de teste	test/architecture.test.ts em cada backend
Comando	npm test (tudo) ou npm run test:arch (só arquitetura)
Total de regras	54 asserções (26 no bff, 12 na Function, 8 em cada serviço)

As regras cobrem exatamente a direção de dependência da clean architecture e o isolamento das fatias:

Regra verificada	O que a asserção garante
domain não depende de application, infrastructure nem presentation	O núcleo é puro: nenhum arquivo de domínio importa camada externa.
application não depende de infrastructure nem de presentation	Casos de uso só conhecem portas e domínio.
presentation não depende de infrastructure	A borda HTTP usa a porta (interface), nunca o adaptador concreto.
Nenhum slice acessa infrastructure/presentation de outro slice	Fronteira vertical: avaliações não importa a infraestrutura de mentores, e vice-versa.
shared não conhece nenhum slice	O kernel compartilhado permanece genérico.
domain não importa frameworks (Express, Zod, driver)	Inspeção de conteúdo: domínio sem dependência de biblioteca.
Sem dependências circulares em src	Proteção contra ciclos introduzidos por refatoração futura.

Trecho real do teste do mentores-service:

```
import { projectFiles } from "archunit";

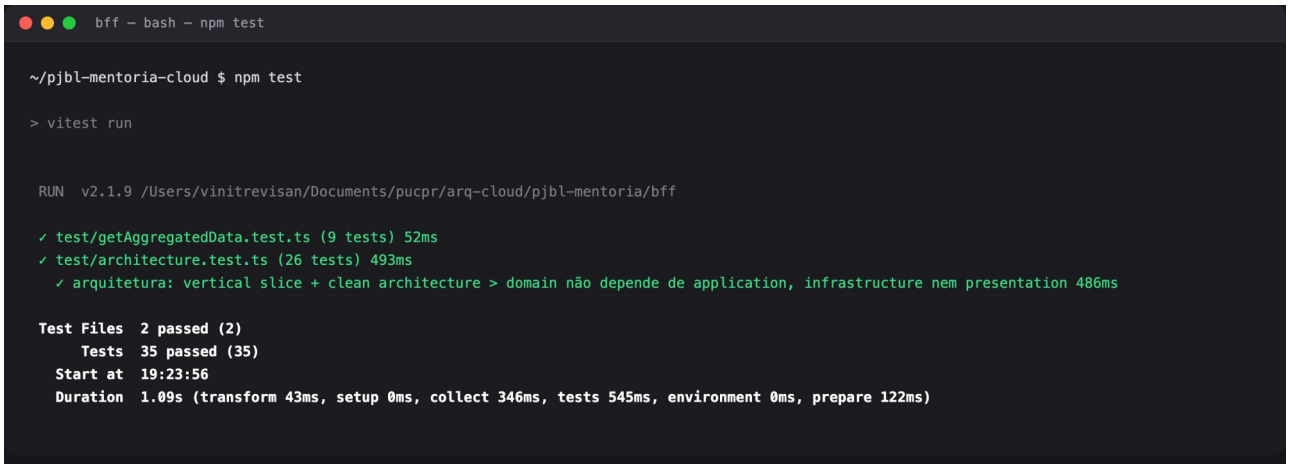
it("domain não depende de application, infrastructure nem presentation", async () => {
  const regra = projectFiles("tsconfig.json")
    .inPath(/^src\/features\/[^/]+\/domain\//)
    .shouldNot()
    .dependOnFiles()
    .inPath(/^src\/features\/[^/]+\/(application|infrastructure|presentation)\//);

  await expect(regra).toPassAsync();
});
```

Observação: o dependency-cruiser usado nas entregas anteriores foi removido. O ArchUnitTS cobre as mesmas regras (e mais), e ter duas ferramentas verificando a mesma fronteira só aumentaria a manutenção. As regras antigas estão reescritas na forma de testes e passam a rodar junto do npm test de cada projeto.

6. Print screens da execução dos testes

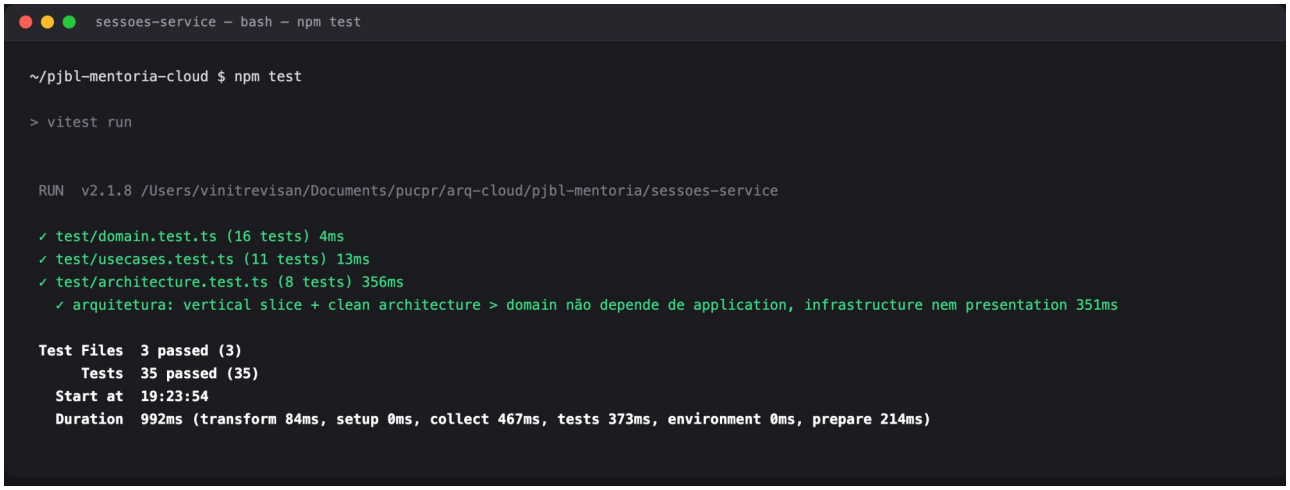
Capturas da saída real do terminal nas quatro pastas, executadas no estado atual da branch. O comando npm test roda os testes de domínio, de casos de uso e os de arquitetura.



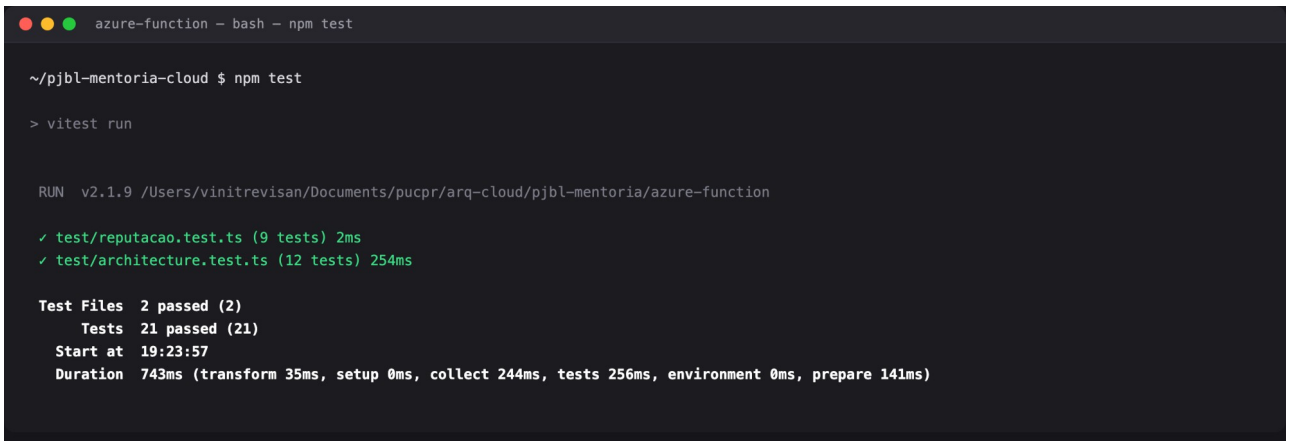
bff — 35 testes, incluindo 26 regras de arquitetura (ArchUnitTS).



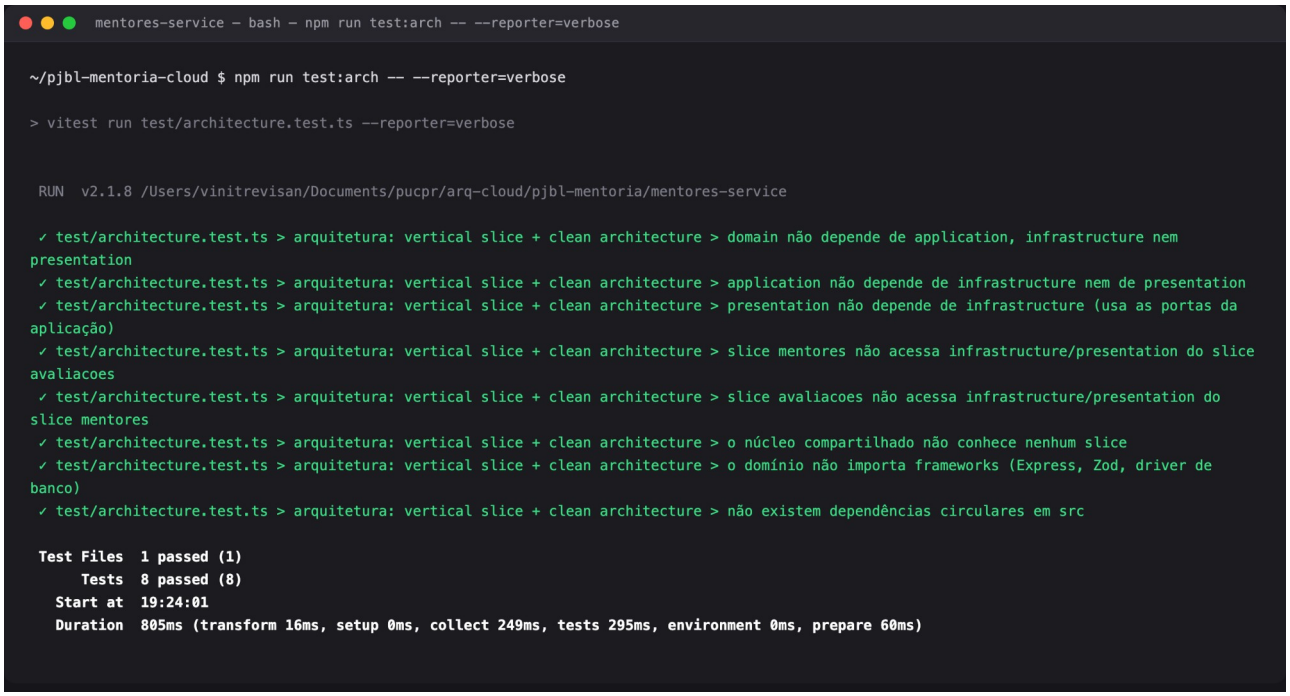
mentores-service — 35 testes, incluindo 8 regras de arquitetura.



sessoes-service — 35 testes, incluindo 8 regras de arquitetura.

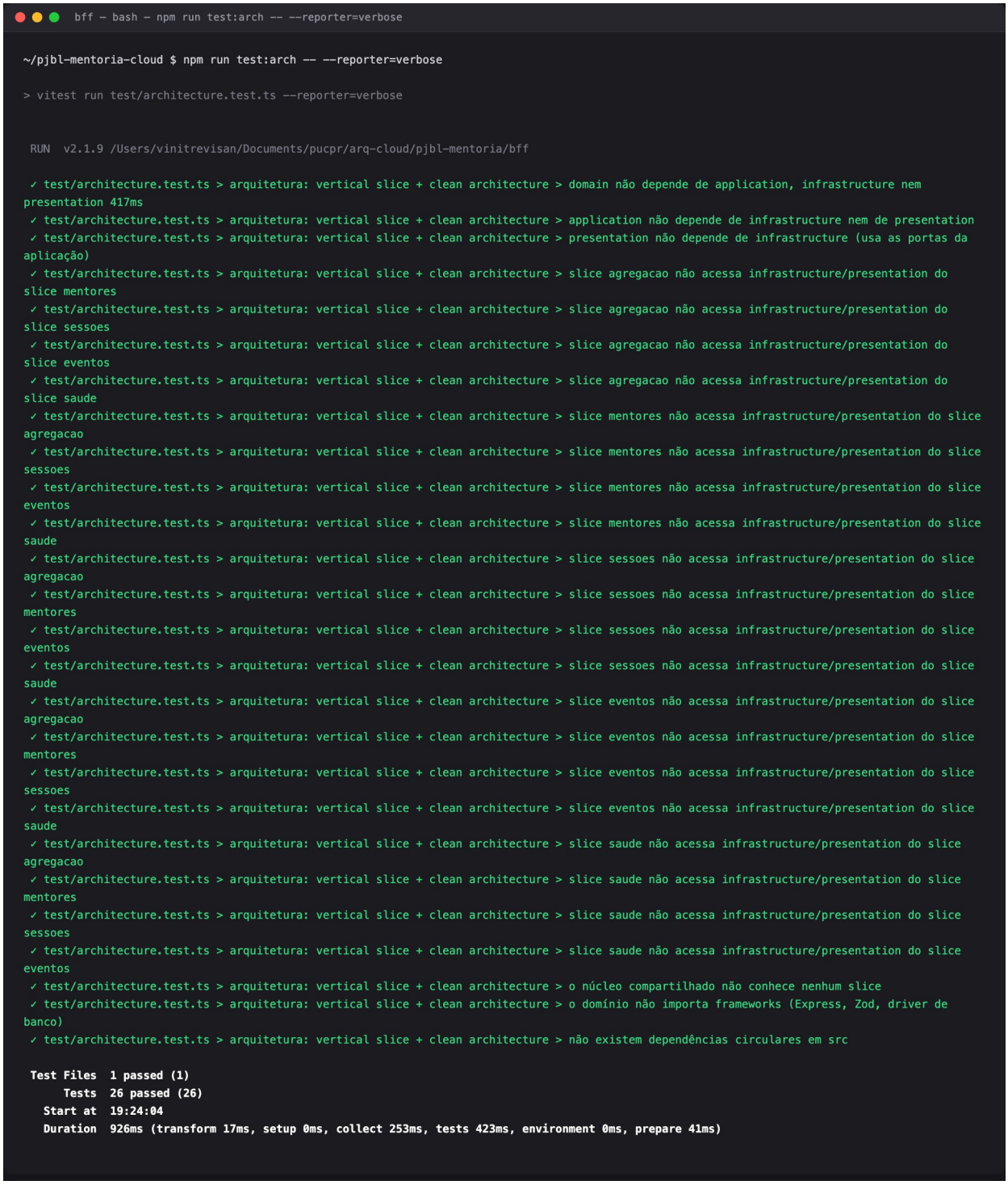


azure-function — 21 testes, incluindo 12 regras de arquitetura.



mentores-service — npm run test:arch: as 8 regras aprovadas.

Acima, a execução detalhada das regras do mentores-service (npm run test:arch). Abaixo, o bff, onde aparecem também as asserções de isolamento entre os cinco slices. O nome de cada asserção é o texto da regra descrita na seção 5.



bff — npm run test:arch: 26 regras aprovadas, incluindo isolamento entre os 5 slices.

7. Como reproduzir

Em qualquer um dos quatro projetos backend:

```
npm install
npm test          # domínio + casos de uso + arquitetura
npm run test:arch # somente as regras de arquitetura
npm run build     # tsc (verificação de tipos)
```

A partir da raiz do repositório, npm run test:all executa os testes dos backends e dos microfrontends; npm run build:all compila todos os projetos.

8. Resultado

- **Vertical slice** : cada capacidade de negócio virou uma pasta autocontida, com as camadas da clean architecture dentro.
- **Inversão de dependência** : as quatro dependências apontam para dentro, e a borda só conhece portas.
- **Fronteira protegida por teste** : 54 asserções ArchUnitTS falham o build se alguém violar a fronteira no futuro.
- **Verificação** : 126 testes verdes e tsc sem erro nos quatro projetos.

O efeito para a manutenção é direto: pedir uma mudança em avaliação de mentor significa abrir uma pasta, e a mudança não alcança o resto do serviço sem que um teste de arquitetura reclame.

9. Referências

- MARTIN, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
- BOGARD, Jimmy. Vertical Slice Architecture. Jimmy Bogard's Blog, 2018.
- ArchUnitTS — documentação e repositório: github.com/LukasNiessen/ArchUnitTS (pacote npm archunit 2.5.4).
- ArchUnit — Architecture Testing for Java: www.archunit.org.
- Vitest — documentação oficial: vitest.dev.